Aplicações de Processamento Digital de Sinais – 4456S-04

Prática 8: Filtros Adaptativos – LMS, NLMS e RLS

Antônio Vinicius Corrêa dos Santos

13/10/2026

1 Configuração

Os três algoritmos foram reunidos em um único `main.m`, com os mesmos parâmetros dos *scripts* de exemplo (`lms.m`, `nlms.m`, `rls.m`):

- Entrada $x = 1{,}74\,\mathcal{U}(-1, 1)$, de variância $\sigma_x^2 = 1{,}74^2/3 \approx 1{,}009$; ruído de medição gaussiano com $\sigma_n^2 = 0{,}001$ ($-30\,\text{dB}$).
- MSE estimado pela média de um *ensemble* de E realizações. Cada realização usa a mesma semente em todas as configurações, então as diferenças entre curvas vêm só do algoritmo e dos parâmetros.
- Sistemas desconhecidos dos exemplos:

$$H_A(z) = \frac{1 - z^{-12}}{1 - z^{-1}} = \sum_{k=0}^{11} z^{-k} \qquad (\texttt{lms.m}),$$

$$H_B(z) = \frac{1 - z^{-12}}{1 + z^{-1}} = \sum_{k=0}^{11} (-1)^k z^{-k} \qquad (\texttt{nlms.m}, \texttt{rls.m}).$$

  O polo cancela um dos zeros de $1 - z^{-12}$, então os dois são FIR de 12 coeficientes (± 1): H_A é um passa-baixas e H_B um passa-altas. Um filtro adaptativo de ordem ord ≥ 11 consegue representá-los exatamente.

Métricas. Para cada configuração são medidos:

- **MSE final**: média do MSE nos últimos 10 % das amostras;
- t_c: tempo de convergência, a primeira amostra a partir da qual a média das 50 seguintes fica a menos de 3 dB do MSE final;
- **erro dos coeficientes**: $\|\bar{w} - h\|^2 / \|h\|^2$ em dB, com $\bar{w}$ a média dos coeficientes finais no *ensemble*;
- **desajuste** $M = (\text{MSE} - \xi_{\min})/\xi_{\min}$, experimental e teórico. O MSE mínimo é $\xi_{\min} = \sigma_n^2 + \sigma_x^2 \sum_{k>\text{ord}} h_k^2$: só o ruído quando ord ≥ 11, e o ruído mais os coeficientes que o filtro não alcança quando a ordem é insuficiente ($\xi_{\min} = 9{,}07\,\text{dB}$ para ord $= 3$ e $6{,}06\,\text{dB}$ para ord $= 7$).

Os parâmetros são variados com ord $= 11$ (a ordem exata); a ordem é variada com o parâmetro padrão de cada *script*. Em todas as figuras, à esquerda está o MSE em dB (média móvel de 25 amostras, eixo de tempo logarítmico) e, à direita, a resposta em frequência do sistema (tracejado) e dos filtros obtidos.

2 LMS (`lms.m`)

$\mathbf{w}(k+1) = \mathbf{w}(k) + 2\mu\, e(k)\, \mathbf{x}(k)$, com $\mu_{\max} = 1/\mathrm{tr}(\mathbf{R}) = 1/((\mathrm{ord}+1)\sigma_x^2)$, sistema H_A, $N = 3000$, $E = 20$. Desajuste teórico (eq. 3.47): $M = \mu\,\mathrm{tr}(\mathbf{R})/(1-\mu\,\mathrm{tr}(\mathbf{R}))$.

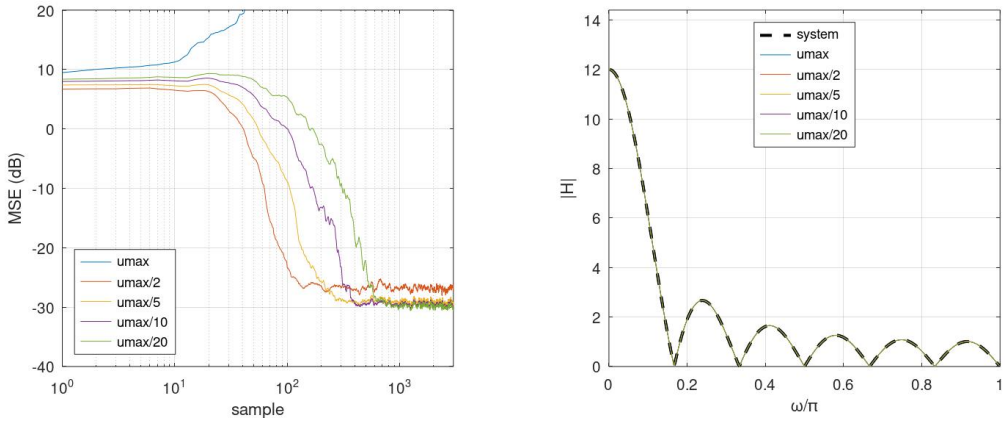


Figura 1: LMS, ord = 11, variando μ.

Tabela 1: LMS, ord = 11, variando μ.

μ	**MSE (dB)**	t_c	**Erro coef. (dB)**	M_{exp}	M_t
$\mu_{\max}$	diverge	—	diverge	—	∞
$\mu_{\max}/2$	−26,63	87	−54,3	1,17	1,00
$\mu_{\max}/5$	−28,98	180	−60,0	0,265	0,250
$\mu_{\max}/10$	−29,53	297	−63,5	0,115	0,111
$\mu_{\max}/20$	−29,78	539	−67,3	0,052	0,053

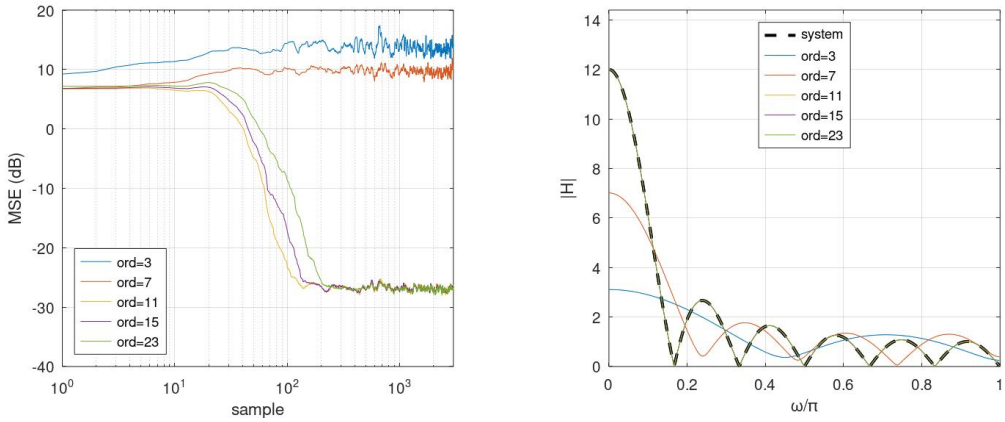


Figura 2: LMS, $\mu = \mu_{\max}/2$, variando a ordem.

Tabela 2: LMS, $\mu = \mu_{\max}(\text{ord})/2$, variando a ordem.

ord	**MSE (dB)**	$\xi_{\min}$ **(dB)**	t_c	**Erro coef. (dB)**	M_{exp}
3	13,96	9,07	—	−1,6	2,08
7	9,98	6,06	—	−4,5	1,47
11	−26,63	−30,00	87	−54,3	1,17
15	−26,67	−30,00	111	−55,7	1,15
23	−26,78	−30,00	163	−53,0	1,10

- **Passo.** $\mu_{\max}$ é o limite de estabilidade: com $\mu = \mu_{\max}$ o desajuste teórico é infinito e o filtro de fato diverge (os coeficientes chegam a 10^{18}). Abaixo dele, o compromisso clássico aparece com clareza: dividir μ por 2 dobra aproximadamente t_c e divide o desajuste por 2. Com $\mu_{\max}/20$ o MSE fica a 0,2 dB do ruído, mas leva 6 vezes mais tempo que $\mu_{\max}/2$. O desajuste experimental concorda com a eq. (3.47) para μ pequeno; em $\mu_{\max}/2$ a aproximação já subestima um pouco (1,17 contra 1,00).
- **Ordem.** Com ord ≥ 11 o sistema é identificado: a resposta em frequência coincide com a do sistema e os coeficientes excedentes vão a zero. A ordem maior não melhora o MSE, só atrasa a convergência (87 → 163 amostras), porque $\mu_{\max}$ cai com $1/(\text{ord}+1)$ e há mais coeficientes a ajustar. Com ord = 3 ou 7 (o padrão do `lms.m`) o filtro não identifica o sistema: o MSE nem desce, e fica acima de $\xi_{\min}$ (com ord = 3 acaba maior que o erro do filtro nulo, 10,8 dB). A parte não modelada age como um ruído de potência $\sim$4–8, e com o passo $\mu_{\max}/2$ o desajuste sobre esse nível é grande. Por isso t_c não se aplica.

3 NLMS

3.1 `nlms.m`: LMS com sinal do erro

O *script* fornecido não implementa o NLMS: a atualização é $\mathbf{w}(k+1) = \mathbf{w}(k) + 2\mu\,\text{sign}(e(k))\,\mathbf{x}(k)$, que é o algoritmo LMS com sinal do erro (*sign-error*), e tanto $\mu_{\max} = \sqrt{\pi\sigma_n^2/2}/\text{tr}(\mathbf{R})$ quanto o desajuste teórico $M = \mu\,\text{tr}(\mathbf{R})\sqrt{\pi/(2\xi_{\min})}$ (eq. 4.28) são os desse algoritmo. Ele foi executado como fornecido (sistema H_B, $N = 10000$, $E = 10$), e um NLMS de verdade foi implementado na Seção 3.2.

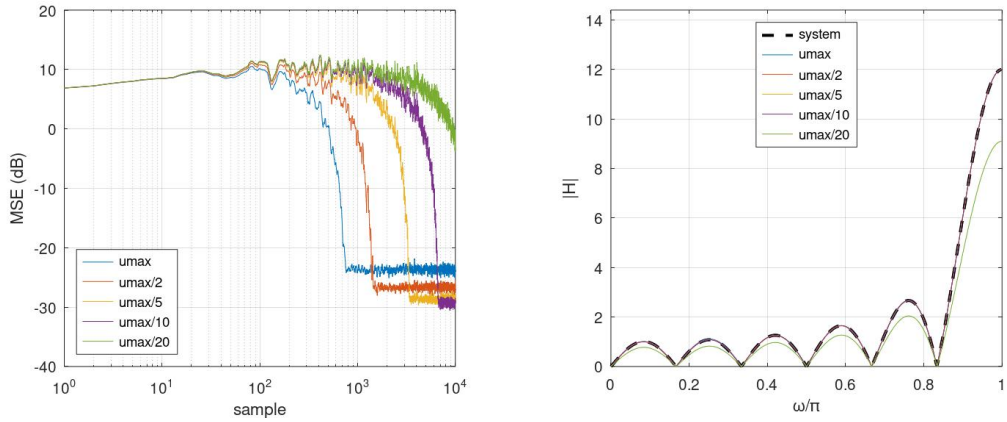


Figura 3: *Sign-error* LMS (`nlms.m`), ord = 11, variando μ.

Tabela 3: *Sign-error* LMS, ord = 11, variando μ.

μ	**MSE (dB)**	t_c	**Erro coef. (dB)**	M_{exp}	M_t
$\mu_{\max}$	$-23{,}74$	709	$-47{,}1$	3,23	1,57
$\mu_{\max}/2$	$-26{,}70$	1369	$-49{,}8$	1,14	0,785
$\mu_{\max}/5$	$-28{,}62$	3332	$-55{,}1$	0,373	0,314
$\mu_{\max}/10$	$-29{,}30$	6568	$-59{,}1$	0,174	0,157
$\mu_{\max}/20$	não converge em 10000 amostras				

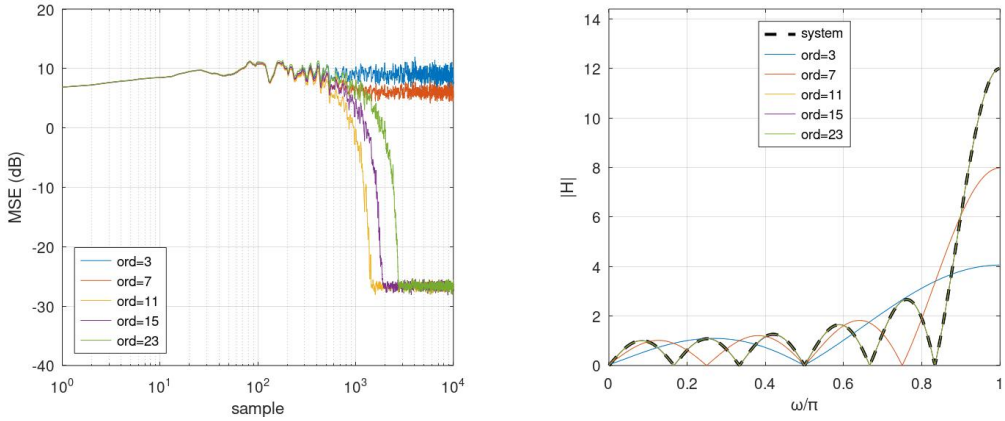


Figura 4: *Sign-error* LMS, $\mu = \mu_{\max}/2$, variando a ordem.

Tabela 4: *Sign-error* LMS, $\mu = \mu_{\max}(\text{ord})/2$, variando a ordem.

ord	**MSE (dB)**	$\xi_{\min}$ **(dB)**	t_c	**Erro coef. (dB)**	M_{exp}
3	8,91	9,07	—	$-1{,}8$	≈ 0
7	6,01	6,06	—	$-4{,}8$	≈ 0
11	$-26{,}70$	$-30{,}00$	1369	$-49{,}8$	1,14
15	$-26{,}67$	$-30{,}00$	1828	$-48{,}4$	1,15
23	$-26{,}68$	$-30{,}00$	2684	$-51{,}2$	1,15

- **Passo.** O $\mu_{\max}$ desse algoritmo é ~ 25 vezes menor que o do LMS, e como o tamanho do passo não depende de $|e|$, a convergência é lenta e quase linear: o MSE fica perto do valor inicial por centenas de amostras e só então cai bruscamente. t_c é proporcional a $1/\mu$ (709 $\rightarrow$ 6568) e com $\mu_{\max}/20$ o filtro não chega a convergir em 10000 amostras (a resposta em frequência ainda está abaixo da do sistema). O desajuste experimental fica sistematicamente acima da eq. (4.28), como no comentário do próprio *script* ($M_t = 0{,}785$, $M_{\text{exp}} = 1{,}27$).
- **Ordem.** Igual ao LMS: identifica para ord ≥ 11, a ordem maior só atrasa. Com ordem insuficiente o MSE fica praticamente sobre $\xi_{\min}$ (desvios de $\pm 4\,\%$, da ordem da incerteza da estimativa): como o erro não modelado é grande, o $\mu_{\max}$ calculado com σ_n^2 é pequeno demais para gerar desajuste.

3.2 NLMS

$\mathbf{w}(k+1) = \mathbf{w}(k) + \dfrac{\mu}{\epsilon + \|\mathbf{x}(k)\|^2}\, e(k)\, \mathbf{x}(k)$, estável para $0 < \mu < 2$; foi usado $\mu_{\max} = 2$, sistema H_B, $N = 3000$, $E = 20$. Desajuste teórico para entrada branca: $M \approx \mu/(2-\mu)$.

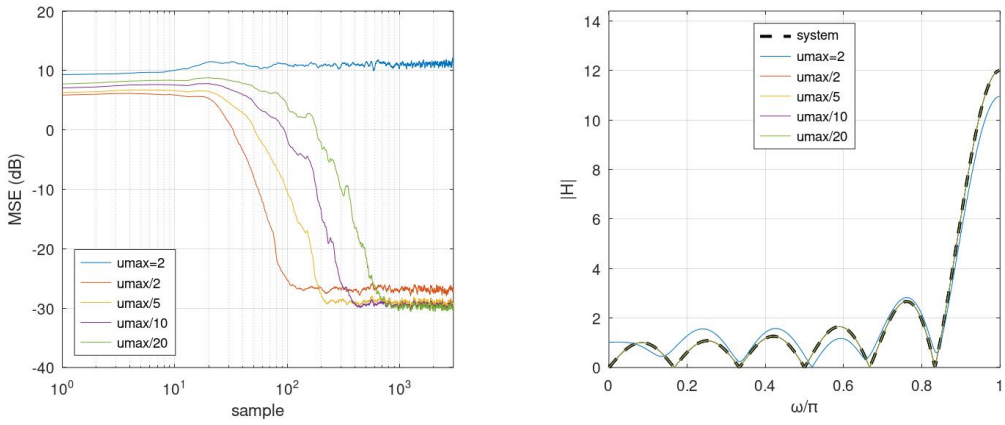


Figura 5: NLMS, ord = 11, variando μ.

Tabela 5: NLMS, ord = 11, variando μ.

μ	**MSE (dB)**	t_c	**Erro coef. (dB)**	M_{exp}	M_t
$\mu_{\max} = 2$	11,29	—	−14,4	$\gg 1$	∞
$\mu_{\max}/2 = 1$	−26,77	70	−57,0	1,11	1,00
$\mu_{\max}/5$	−28,94	165	−61,0	0,276	0,250
$\mu_{\max}/10$	−29,51	301	−63,8	0,120	0,111
$\mu_{\max}/20$	−29,77	589	−67,5	0,053	0,053

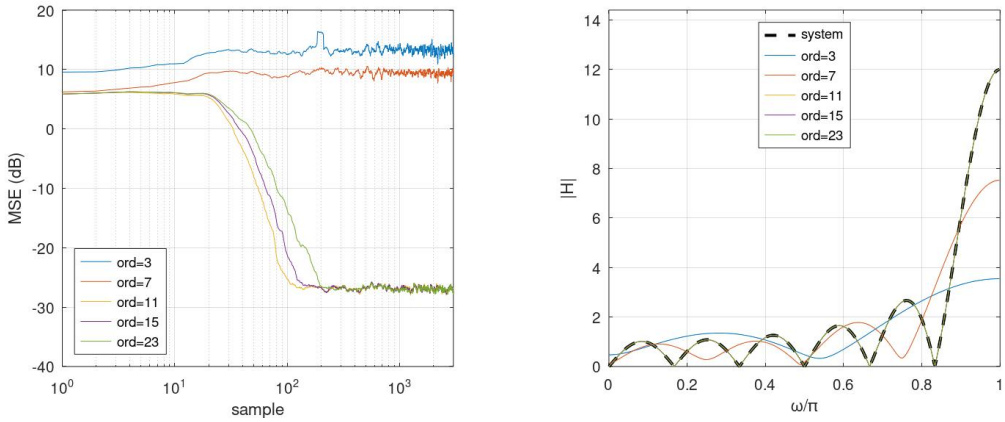


Figura 6: NLMS, $\mu = 1$, variando a ordem.

Tabela 6: NLMS, $\mu = 1$, variando a ordem.

ord	**MSE (dB)**	$\xi_{\min}$ **(dB)**	t_c	**Erro coef. (dB)**	M_{exp}
3	13,33	9,07	—	−1,6	1,67
7	9,38	6,06	—	−4,7	1,15
11	−26,77	−30,00	70	−57,0	1,11
15	−26,79	−30,00	97	−55,6	1,10
23	−26,87	−30,00	153	−53,3	1,06

- Com entrada branca e de potência constante, $\|\mathbf{x}\|^2 \approx (\text{ord} + 1)\sigma_x^2$, e o NLMS se comporta

como um LMS com $\mu_{\text{LMS}} = \mu/(2\,\text{tr}(\mathbf{R}))$. Por isso a tabela é quase igual à do LMS: mesmo compromisso velocidade × desajuste, e M_{exp} segue $\mu/(2-\mu)$. A vantagem do NLMS (independência da potência do sinal, convergência melhor com entrada colorida) não aparece nesse cenário.

- No limite $\mu = 2$ o filtro não diverge, mas também não converge: o MSE fica em 11 dB e a resposta em frequência é só uma aproximação grosseira.
- Ordem: mesmas conclusões do LMS.

4 RLS (`rls.m`)

Sistema H_B, $N = 2000$, $E = 10$, $\mathbf{S}_D(0) = \delta\mathbf{I}$ com $\delta = 1/\sigma_x^2$. O *script* calcula o erro *a posteriori* (depois de atualizar $\mathbf{w}$), que fica abaixo do nível de ruído e não é comparável ao dos outros algoritmos; aqui foi usado o erro *a priori*, como nos LMS. Desajuste teórico: $M = (\text{ord}+1)(1-\lambda)/(1+\lambda)$.

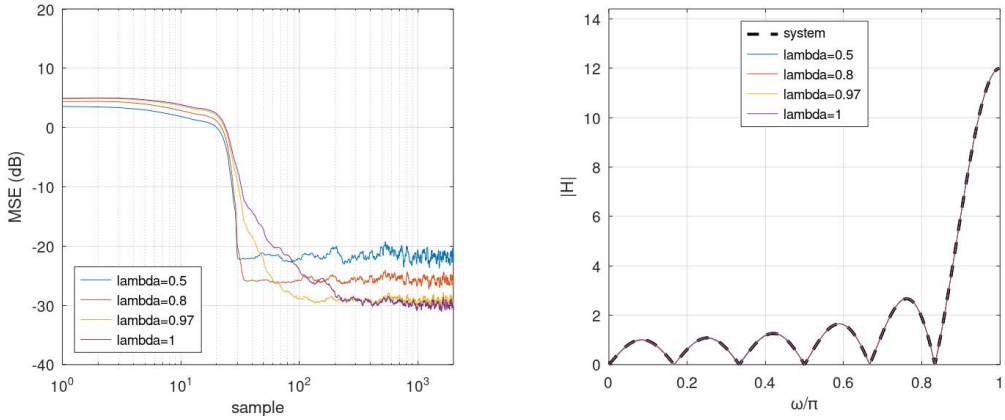


Figura 7: RLS, ord = 11, variando λ.

Tabela 7: RLS, ord = 11, variando λ.

λ	**MSE (dB)**	t_c	**Erro coef. (dB)**	M_{exp}	M_t
0,5	−21,81	18	−45,8	5,59	4,00
0,8	−25,78	19	−51,0	1,64	1,33
0,97	−29,18	46	−57,9	0,209	0,183
1	−29,91	159	−64,7	0,020	$\to 0$

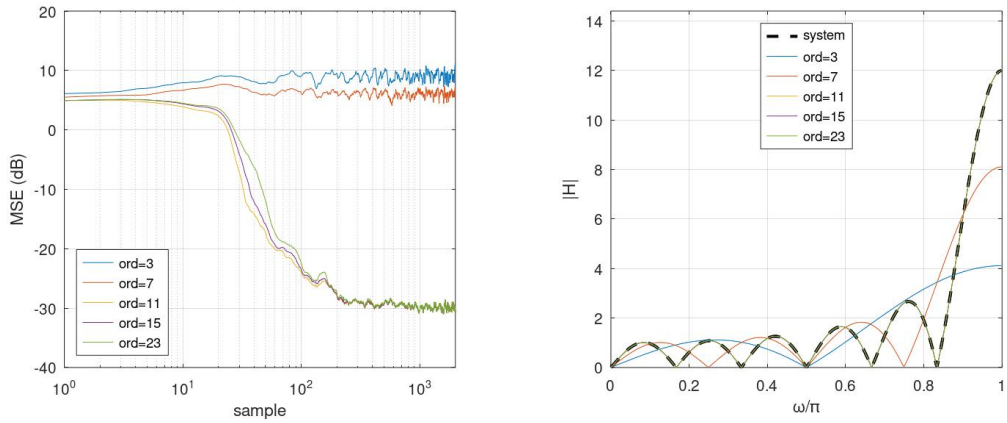


Figura 8: RLS, $\lambda = 1$, variando a ordem.

Tabela 8: RLS, $\lambda = 1$, variando a ordem.

ord	**MSE (dB)**	$\xi_{\min}$ **(dB)**	t_c	**Erro coef. (dB)**	M_{exp}
3	9,36	9,07	—	−1,8	0,068
7	6,25	6,06	—	−4,8	0,045
11	−29,91	−30,00	159	−64,7	0,020
15	−29,90	−30,00	161	−64,3	0,023
23	−29,88	−30,00	165	−64,0	0,027

- **Fator de esquecimento.** λ define uma memória efetiva de $1/(1-\lambda)$ amostras. Com $\lambda = 0{,}5$ ela é de 2 amostras para estimar 12 coeficientes: o filtro converge rápido (18 amostras) mas o MSE fica 8 dB acima do ruído. Aumentar λ alonga a memória e reduz o desajuste até $\lambda = 1$, que é a solução de mínimos quadrados sobre todo o histórico e chega praticamente ao nível de ruído ($M_{\text{exp}} = 0{,}02$). Os três valores pedidos identificam o sistema; o que muda é a precisão. $\lambda < 1$ só se justifica se o sistema variasse no tempo, o que não é o caso aqui.
- **Ordem.** Também identifica a partir de ord = 11, e diferente do LMS a ordem maior quase não afeta nem o tempo nem o MSE. Com ordem insuficiente o RLS fica sobre $\xi_{\min}$: ele converge para a solução de Wiener do filtro truncado, sem o desajuste grande do LMS.
- **Inicialização.** As 159 amostras para $\lambda = 1$ ficam bem acima das $\sim 2(\text{ord}+1)$ esperadas para o RLS. A causa é $\delta = 1/\sigma_x^2$: $\mathbf{S}_D(0) = \delta\mathbf{I}$ equivale a uma estimativa inicial $\mathbf{R}(0) = \mathbf{I}/\delta$, cujo viés só decai como $1/k$. Com $\delta = 100$ (prior fraco) o mesmo filtro converge em 18 amostras (Figura 9).

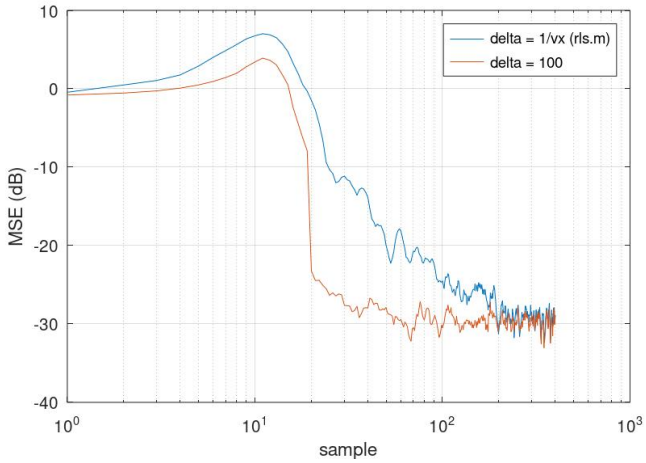


Figura 9: RLS, $\lambda = 1$, ord $= 11$: inicialização do *script* ($\delta = 1/\sigma_x^2$, $t_c = 124$) e $\delta = 100$ ($t_c = 18$), primeiras 400 amostras.

5 Comparação

Os quatro algoritmos foram executados no mesmo sistema (H_B), com ord $= 11$, $N = 10000$, $E = 10$ e o parâmetro padrão de cada *script*.

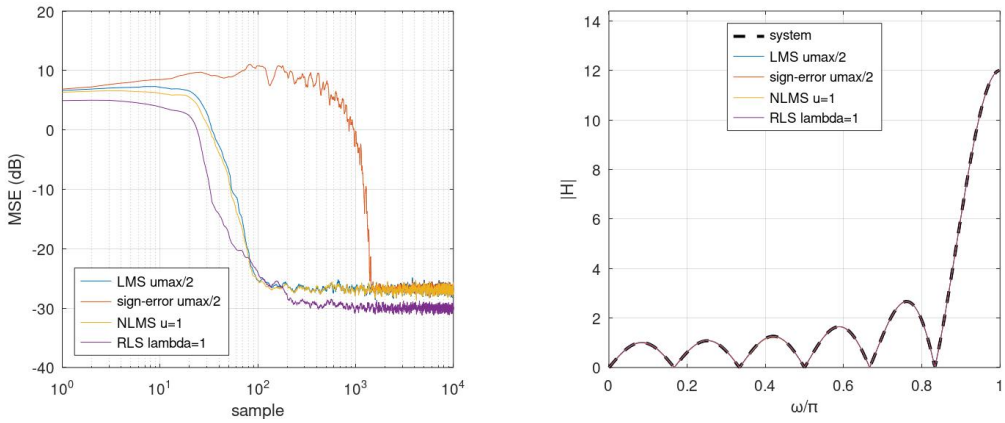


Figura 10: Comparação dos algoritmos no mesmo sistema.

Tabela 9: Comparação, ord $= 11$, sistema H_B.

Algoritmo	**MSE (dB)**	t_c	**Erro coef. (dB)**
LMS, $\mu_{\max}/2$	$-26{,}93$	74	$-49{,}8$
Sign-error (`nlms.m`), $\mu_{\max}/2$	$-26{,}70$	1369	$-49{,}8$
NLMS, $\mu = 1$	$-26{,}97$	71	$-50{,}3$
RLS, $\lambda = 1$	$-30{,}02$	162	$-76{,}4$

- **Menor ordem.** Todos identificam o sistema a partir de ord $= 11$ e nenhum com menos. A ordem mínima é uma propriedade do sistema (12 coeficientes), não do algoritmo; abaixo dela todos ficam limitados por $\xi_{\min}$, e o RLS e o *sign-error* chegam mais perto desse limite.
- **Tempo de convergência.** Com os parâmetros padrão, LMS e NLMS convergem em ~ 70 amostras e o *sign-error* em ~ 1400, quase 20 vezes mais lento. O RLS com $\lambda = 1$ leva

$\sim$160 só por causa da inicialização do *script*; com δ grande, ou com $\lambda < 1$, converge em $\sim$18 amostras, o mais rápido de todos.

- **MSE mínimo.** O RLS com $\lambda = 1$ é o único que chega ao nível de ruído ($-30{,}0$ dB, desajuste ≈ 0) e tem o menor erro de coeficientes por uma margem grande (-76 dB contra -50 dB). Os LMS ficam $\sim$3 dB acima do ruído com $\mu_{\max}/2$; podem chegar a $-29{,}8$ dB com $\mu_{\max}/20$, mas aí levam $\sim$550 amostras.

6 Conclusões

O RLS teve o melhor desempenho na identificação: o menor MSE, os coeficientes mais precisos e, com inicialização adequada, a convergência mais rápida, e sem o compromisso velocidade $\times$ desajuste que limita os LMS. O custo é computacional: $O(L^2)$ operações por amostra e uma matriz $L \times L$, contra $O(L)$ do LMS. Entre os LMS, LMS e NLMS foram equivalentes com entrada branca, e o *sign-error* do `nlms.m` foi o mais lento, em troca de não precisar de multiplicação pelo erro. Em todos os algoritmos o passo (ou λ) controla o compromisso entre velocidade e precisão, e a ordem só precisa ser suficiente para representar o sistema: ordem a mais custa tempo de convergência no LMS e quase nada no RLS, e ordem a menos impede a identificação em todos eles.

Script (Octave)

```
close all
clear all
clc

pkg load signal

global VIS = 'off';
global VX  = 1.74^2/3;    % variance of x = 1.74*U(-1,1)
global VN  = 0.001;       % measurement noise variance

if ~exist('plots', 'dir'), mkdir('plots'); end

sysA = struct('b', [1 zeros(1,11) -1], 'a', [1 -1]);    % h = [1 1 ... 1]
sysB = struct('b', [1 zeros(1,11) -1], 'a', [1  1]);    % h = [1 -1 1 ... -1]

% ------------------------------------------------------------------ helpers

function save_fig(tag)
    saveas(gcf, ['plots/' tag '.jpg']);
end

% Runs E realizations of one algorithm and returns the averaged MSE curve and
% final coefficients. p is the step size u, or lambda for 'rls'.
function [mse, w] = adapt(alg, p, ord, sys, N, E, delta)
    global VX VN
    if nargin < 7, delta = 1/VX; end   % initial S = delta*I
    L   = ord + 1;
    mse = zeros(1, N);
    w   = zeros(L, 1);
    for k = 1:E
        % Fixed seed, so every configuration sees the same signals.
        rand('state', k); randn('state', k);
        x  = 1.74 * (2*rand(1,N) - 1);
        d  = filter(sys.b, sys.a, x) + sqrt(VN) * randn(1,N);
        xp = [zeros(1,ord) x];
        W  = zeros(L, 1);
        e  = zeros(1, N);
        switch alg
            case 'lms'
                for j = 1:N
                    X = xp(j+ord:-1:j)';
                    e(j) = d(j) - W'*X;
```

```
                    W = W + 2*p*e(j)*X;
                end
            case 'sign'   % sign-error LMS
                for j = 1:N
                    X = xp(j+ord:-1:j)';
                    e(j) = d(j) - W'*X;
                    W = W + 2*p*sign(e(j))*X;
                end
            case 'nlms'
                for j = 1:N
                    X = xp(j+ord:-1:j)';
                    e(j) = d(j) - W'*X;
                    W = W + p/(eps + X'*X)*e(j)*X;
                end
            case 'rls'
                S  = delta * eye(L);
                pd = zeros(L, 1);
                for j = 1:N
                    X = xp(j+ord:-1:j)';
                    % Error before the update, as in the LMS variants.
                    e(j) = d(j) - W'*X;
                    S  = (S - (S*X*X'*S) / (p + X'*S*X)) / p;
                    pd = p*pd + d(j)X;
                    W  = S*pd;
                end
        end
        mse = mse + e.^2;
        w   = w + W;
    end
    mse = mse / E;
    w   = w / E;
end

% mf:   final MSE (mean of the last 10%)
% tc:   first sample after which the next 50 stay within 3 dB of mf
% ce:   coefficient error in dB
% emin: lowest possible MSE (noise plus the taps beyond the filter length)
function [mf, tc, ce, emin] = metrics(mse, w, sys)
    global VX VN
    N  = numel(mse);
    mf = mean(mse(round(0.9*N):N));
    tc = find(movmean(mse, [0 49]) <= 2*mf, 1);
    if isempty(tc) || ~isfinite(mf), tc = NaN; end
    h  = impz(sys.b, sys.a, 12);
    L  = max(numel(w), 12);
    dw = [w; zeros(L-numel(w),1)] - [h; zeros(L-12,1)];
    ce = 10*log10(sum(dw.^2) / sum(h.^2));
    emin = VN + VX*sum(h(numel(w)+1:end).^2);
end

% Runs each [param ord] row of cfg, prints a table and saves the MSE curves and
% frequency responses. Mt gives the theoretical misadjustment.
function sweep(fig_id, tag, alg, cfg, labels, sys, N, E, Mt)
    global VIS VN
    figure(fig_id, 'visible', VIS);
    [H, wf] = freqz(sys.b, sys.a, 512);
    subplot(1,2,2); plot(wf/pi, abs(H), 'k--', 'linewidth', 2); hold on;
    printf('\n--- %s ---\n%-14s %10s %9s %10s %8s %8s %8s\n', tag, 'config', ...
           'MSE (dB)', 'tc', 'coef (dB)', 'Mexp', 'Mt', 'emin dB');
    for i = 1:rows(cfg)
        [mse, w] = adapt(alg, cfg(i,1), cfg(i,2), sys, N, E);
        [mf, tc, ce, emin] = metrics(mse, w, sys);
        printf('%-14s %10.2f %9d %10.2f %8.3g %8.3g %8.2f\n', labels{i}, ...
               10*log10(mf), tc, ce, (mf - emin)/emin, Mt(cfg(i,1), cfg(i,2), emin), ...
               10*log10(emin));
        subplot(1,2,1); semilogx(10*log10(movmean(mse, 25))); hold on;
        subplot(1,2,2); plot(wf/pi, abs(freqz(w, 1, 512)));
    end
    % Fixed limits so a diverging run doesn't stretch the axes.
    subplot(1,2,1); grid on; xlim([1 N]); ylim([-40 20]);
    xlabel('sample'); ylabel('MSE (dB)'); legend(labels(:)', 'location', 'southwest');
    subplot(1,2,2); grid on; xlim([0 1]); ylim([0 1.2*max(abs(H))]);
```

```
    xlabel('\omega/\pi'); ylabel('|H|'); legend(['system' labels(:)'], 'location', 'north');
    set(gcf, 'paperposition', [0 0 11 4]);
    save_fig(tag);
end

no_mt = @(p, ord, emin) NaN;

% ================================================================ LMS

N = 3000; E = 20;
umax = @(ord) 1 / ((ord+1)*VX);
Mt   = @(u, ord, emin) u*(ord+1)*VX / (1 - u*(ord+1)*VX);

k = [1 2 5 10 20];
sweep(1, 'lms_u', 'lms', [umax(11)./k' 11*ones(5,1)], ...
      {'umax', 'umax/2', 'umax/5', 'umax/10', 'umax/20'}, sysA, N, E, Mt);

o = [3 7 11 15 23]';
sweep(2, 'lms_ord', 'lms', [arrayfun(umax, o)/2 o], ...
      arrayfun(@(n) sprintf('ord=%d', n), o, 'uniformoutput', false), sysA, N, E, Mt);

% ============================================================ sign-error LMS (nlms.m)

N = 10000; E = 10;
umax = @(ord) sqrt(pi*VN/2) / ((ord+1)*VX);
Mt   = @(u, ord, emin) u*(ord+1)*VX*sqrt(pi/(2*emin));

sweep(3, 'sign_u', 'sign', [umax(11)./k' 11*ones(5,1)], ...
      {'umax', 'umax/2', 'umax/5', 'umax/10', 'umax/20'}, sysB, N, E, Mt);
sweep(4, 'sign_ord', 'sign', [arrayfun(umax, o)/2 o], ...
      arrayfun(@(n) sprintf('ord=%d', n), o, 'uniformoutput', false), sysB, N, E, Mt);

% ================================================================ NLMS

N = 3000; E = 20;
Mt = @(u, ord, emin) u / (2 - u);   % approximation for white input

sweep(5, 'nlms_u', 'nlms', [2./k' 11*ones(5,1)], ...
      {'umax=2', 'umax/2', 'umax/5', 'umax/10', 'umax/20'}, sysB, N, E, Mt);
sweep(6, 'nlms_ord', 'nlms', [ones(5,1) o], ...
      arrayfun(@(n) sprintf('ord=%d', n), o, 'uniformoutput', false), sysB, N, E, Mt);

% ================================================================ RLS

N = 2000; E = 10;
Mt = @(l, ord, emin) (ord+1)*(1-l) / (1+l);

sweep(7, 'rls_lambda', 'rls', [0.5 11; 0.8 11; 0.97 11; 1 11], ...
      {'lambda=0.5', 'lambda=0.8', 'lambda=0.97', 'lambda=1'}, sysB, N, E, Mt);
sweep(8, 'rls_ord', 'rls', [ones(5,1) o], ...
      arrayfun(@(n) sprintf('ord=%d', n), o, 'uniformoutput', false), sysB, N, E, Mt);

% Effect of the initialization: delta = 1/vx (rls.m) against a large delta.
figure(10, 'visible', VIS);
for delta = [1/VX 100]
    [mse, w] = adapt('rls', 1, 11, sysB, 400, E, delta);
    [mf, tc] = metrics(mse, w, sysB);
    printf('RLS delta=%6.2f  tc=%d\n', delta, tc);
    semilogx(10*log10(movmean(mse, 5))); hold on;
end
grid on; xlim([1 400]); xlabel('sample'); ylabel('MSE (dB)');
legend('delta = 1/vx (rls.m)', 'delta = 100');
set(gcf, 'paperposition', [0 0 6 4]);
save_fig('rls_delta');

% ============================================================ comparison

% All algorithms on the same system, ord = 11, default parameters.
N = 10000; E = 10;
sweep(9, 'compare', 'lms', [1/(12*VX)/2 11], {'LMS umax/2'}, sysB, N, E, no_mt);
figure(9, 'visible', VIS);
```

```
algs = {'sign', sqrt(pi*VN/2)/(12*VX)/2, 'sign-error umax/2'; ...
        'nlms', 1,                       'NLMS u=1'; ...
        'rls',  1,                       'RLS lambda=1'};
for i = 1:rows(algs)
    [mse, w] = adapt(algs{i,1}, algs{i,2}, 11, sysB, N, E);
    [mf, tc, ce] = metrics(mse, w, sysB);
    printf('%-18s %10.2f %9d %10.2f\n', algs{i,3}, 10*log10(mf), tc, ce);
    subplot(1,2,1); semilogx(10*log10(movmean(mse, 25)));
    subplot(1,2,2); plot((0:511)/512, abs(freqz(w, 1, 512)));
end
subplot(1,2,1); legend(['LMS umax/2' algs(:,3)'], 'location', 'southwest');
subplot(1,2,2); legend(['system' 'LMS umax/2' algs(:,3)'], 'location', 'north');
save_fig('compare');
```

Listing 1: `main.m`